

SEGMENT TREES

Zero to Hero: fast range queries and updates

Your goal

You will learn **why** a segment tree exists, how to **build**, **query**, **update**, and how lazy propagation makes range updates fast.

Prerequisites: arrays, recursion, and the idea of a binary tree. No prior segment-tree knowledge required.

Study route: 1) ranges - 2) point update - 3) custom node information - 4) lazy propagation - 5) solve progressively.

1. The problem: many questions about one array

Suppose $A = [2, 2, 1, 4, 3, 7, 1, 1]$. A range query asks about a continuous slice, for example $\text{sum}(2..6) = 1+4+3+7+1 = 16$.

- A normal loop answers one range sum in $O(n)$. Fine once; painful for 100,000 queries.
- A prefix-sum array answers $\text{sum}(l..r)$ in $O(1)$, but changing $A[i]$ then costs $O(n)$ to repair all later prefixes.
- We want both query and update to be fast: $O(\log n)$.

The central idea

Store answers for many intervals in a binary tree. Each node represents a segment $[L..R]$. Its two children split it at $M = (L+R)/2$. The parent stores the combination of its children.

For a **sum** tree: parent = left sum + right sum. For a **minimum** tree: parent = $\min(\text{left}, \text{right})$.

[0..7] = 31							
[0..3] = 9							
[0..1] = 4		[4..5] = 10					
2	2	1	4	3	7	1	1

The bottom row is the original array. Every internal cell summarizes the cells beneath it. This is why the data structure is called a **segment** tree.

Why it is fast

At each level, a query visits only the pieces that touch its boundary. The tree has about $\log_2(n)$ levels, so a query/update touches $O(\log n)$ nodes. Building all summaries once costs $O(n)$.

2. The three range-query cases

Every recursive query compares the node segment $[L..R]$ with the requested range $[ql..qr]$.

Case	Meaning	What to return
No overlap	$[L..R]$ is completely outside	Identity value (0 for sum, +infinity for min)
Complete overlap	$[L..R]$ lies fully inside	The value already stored at this node
Partial overlap	They intersect, but neither contains the other	Ask both children and combine their answers

Mental walkthrough: sum(2..6)

At root $[0..7]$, it is partial: split into $[0..3]$ and $[4..7]$. In $[0..3]$, only $[2..3]$ is relevant: get 5. In $[4..7]$, $[4..5]$ is completely inside (10), then from $[6..7]$ only index 6 contributes (1). Combine $5+10+1 = 16$.

Identity values matter

The identity is the neutral value for your merge: sum $\rightarrow 0$; min $\rightarrow +\text{infinity}$; max $\rightarrow -\text{infinity}$; gcd $\rightarrow 0$; xor $\rightarrow 0$. If a branch is outside the query, returning identity makes combining safe.

Never use 0 as the "no overlap" result for a minimum query: $\min(0, \text{real answer})$ would create a wrong answer.

Array implementation

We normally store the tree in an array **tree** of size $4*n$. If a node is at index p , its children are $2*p$ and $2*p+1$. This wastes a little memory but is simple and safe.

3. Core template: Range Sum + Point Update

This directly solves LeetCode 307 / Range Sum Query - Mutable. Keep this template; then change only merge and identity for many variants.

```
class NumArray {
    vector tree; int n;

    void build(int p, int L, int R, vector& a) {
        if (L == R) { tree[p] = a[L]; return; }
        int M = (L + R) / 2;
        build(2*p, L, M, a); build(2*p+1, M+1, R, a);
        tree[p] = tree[2*p] + tree[2*p+1];
    }

    void update(int p, int L, int R, int i, int val) {
        if (L == R) { tree[p] = val; return; }
        int M = (L + R) / 2;
        if (i <= M) update(2*p, L, M, i, val);
        else update(2*p+1, M+1, R, i, val);
        tree[p] = tree[2*p] + tree[2*p+1]; // rebuild upward
    }

    int query(int p, int L, int R, int ql, int qr) {
        if (qr < L || R < ql) return 0;
        if (ql <= L && R <= qr) return tree[p];
        int M = (L + R) / 2;
        return query(2*p, L, M, ql, qr) + query(2*p+1, M+1, R, ql, qr);
    }
public:
    NumArray(vector& a) : n(a.size()), tree(4*n) { build(1,0,n-1,a); }
    void update(int i, int val) { update(1,0,n-1,i,val); }
    int sumRange(int l, int r) { return query(1,0,n-1,l,r); }
};
```

Point update intuition

Only one leaf changes. Recompute every ancestor on the one root-to-leaf path. That path has $O(\log n)$ nodes.

Common indexing bug: this code is 0-indexed. If a problem gives indices 1..n, call build/query/update with 1..n or subtract 1 consistently.

4. Range Minimum Query (RMQ) + Point Update

Same tree, different meaning. Store the minimum of each segment.

```
// Only these lines change from the sum template:  
+// merge:    tree[p] = min(tree[2*p], tree[2*p+1]);  
+// no overlap in query: return INT_MAX;  
+// answer merge: return min(leftAnswer, rightAnswer);
```

Example

For $A = [5, 2, 8, 1, 7]$, $\min(1..3)$ is 1. If you update $A[3] = 6$, only that leaf and its ancestors change; then $\min(1..3)$ becomes 2. A segment tree does not care what you aggregate - it only needs a sensible merge operation and identity.

Quick design checklist

1. What must each node store?
2. How do two child values combine?
3. What is the identity for no overlap?
4. On a leaf update, what is the new stored value?

When a Fenwick tree is enough

For prefix/range sums with point updates, a Fenwick (BIT) is shorter and uses less memory. Use a segment tree when you need min/max/gcd/custom information, or range updates with lazy propagation. Segment trees are more general.

5. From values to richer answers

A node can store a struct, not just one number. The merge function becomes your superpower.

Problem	Node stores	Merge children
Maximum and its frequency	(maxValue, count)	keep larger max; if equal, add counts
Count zeros	number of zeros	left.zeros + right.zeros
Range sum	sum	left.sum + right.sum
Range minimum	minimum	min(left.min, right.min)

Maximum value and frequency: merge function

```
pair merge(pair a, pair b) {
    if (a.first > b.first) return a;
    if (b.first > a.first) return b;
    return {a.first, a.second + b.second};
}
// leaf for value x: {x, 1}
// no overlap: {INT_MIN, 0}
```

Count zeros in a range

Store 1 at a leaf if $A[i]$ is zero; otherwise store 0. Parent = left + right. For a point update, replace that leaf with the new 0/1 value. The range sum of this transformed array is the number of zeros.

Useful viewpoint

Many segment-tree problems become easy when you first ask: “What single summary would let two neighboring intervals combine into the summary for their union?”

6. Why lazy propagation exists

A point update changes one position. But an update like “add 5 to every $A[i]$ in $[l..r]$ ” may change n positions. Visiting all of them loses the benefit of the tree.

Lazy propagation in one sentence

When an update completely covers a node segment, update that node's stored answer now and **postpone** telling its children until somebody needs them.

For range add + range sum, each node needs two arrays:

- **tree[p]**: sum of the segment represented by p.
- **lazy[p]**: an addition that all elements in this segment still need (or whose effect has not been passed to children).

If +x covers $[L..R]$ completely

```
tree[p] += x * (R - L + 1);  
lazy[p] += x;
```

Push before going down

If you need to visit children, first push the pending lazy value from p. Otherwise children are stale.

```
void push(int p, int L, int R) {  
    if (lazy[p] == 0 || L == R) return;  
    int M = (L + R) / 2;  
    long long x = lazy[p];  
    tree[2*p] += x * (M - L + 1);  
    tree[2*p+1] += x * (R - M);  
    lazy[2*p] += x; lazy[2*p+1] += x;  
    lazy[p] = 0;  
}
```

7. Lazy propagation template: range add + range sum

This is the Day 26/27 range-update-and-range-query pattern. Use long long for sums and updates.

```
void rangeAdd(int p, int L, int R, int ql, int qr, long long x) {
    if (qr < L || R < ql) return;
    if (ql <= L && R <= qr) {
        tree[p] += x * (R - L + 1); lazy[p] += x; return;
    }
    push(p, L, R);
    int M = (L + R) / 2;
    rangeAdd(2*p, L, M, ql, qr, x);
    rangeAdd(2*p+1, M+1, R, ql, qr, x);
    tree[p] = tree[2*p] + tree[2*p+1];
}

long long rangeSum(int p, int L, int R, int ql, int qr) {
    if (qr < L || R < ql) return 0;
    if (ql <= L && R <= qr) return tree[p];
    push(p, L, R);
    int M = (L + R) / 2;
    return rangeSum(2*p, L, M, ql, qr) + rangeSum(2*p+1, M+1, R, ql, qr);
}
```

Rule of thumb

Before descending into children in a lazy tree, push. **After updating children**, pull/rebuild the parent from its children.

What changes for other operations?

For range add + range maximum, full cover is **tree[p] += x** (no length multiplier), and parent merge is max. For assignment updates, lazy needs a “set pending” tag rather than simply adding values. The idea remains: store deferred work at the highest fully covered node.

8. Practice map for your sheet

Solve in this order. Do not rush into lazy propagation before you can trace a normal query by hand.

Stage	Your problem	Target skill	Definition of done
1	Range Sum Query - Mutable	Build, query, point set update	Pass LeetCode 307; explain the 3 overlap cases
2	Range Minimum Query and Point Update	Change merge + identity	Implement from blank without copying sum code
3	Range Update and Range Query	Lazy range add + range sum	Explain why length is multiplied for sums
4	Maximum Element and its Frequency	Pair node + custom merge	Write merge and identity before tree code
5	Count Zeros in a Range	Transform to 0/1 summary	Handle updates and queries
6	Block Placement Queries	Advanced: coordinate thinking + max gap summary	First understand constraints; isolate node invariant

A problem-solving ritual

- Write 5-8 array values. Draw 2-3 tree levels. Trace one query.
- State node meaning in one sentence. Example: "tree[p] is the sum of A[L..R]."
- Write merge, identity, and leaf value before writing recursion.
- Test: $n=1$, query entire array, query one index, update first/last index, negative values (especially for max/min).

For Block Placement Queries

Treat it as an advanced application, not a starting point. The key is usually to maintain information about gaps between obstacles and query a maximum relevant gap. Learn the first five problems first; this one becomes much less mysterious.

9. Bugs that catch almost everyone

Save yourself debugging time with this compact checklist.

Symptom	Likely cause	Fix
Wrong minimum answer	No-overlap returned 0	Return INT_MAX
Lazy answers wrong after mixed ops	Forgot push before descending	Call push in partial query/update
Range sum too small after lazy add	Forgot segment length	Add $x * (R-L+1)$
Random wrong results / overflow	int is too small	Use long long for sums and lazy
Last index ignored	Wrong split or ranges	Left [L..M], right [M+1..R]
Works except n=1	Base case missing	if (L == R) is leaf

Complexity cheat sheet

Operation	Time	Why
Build	$O(n)$	Each tree node is built once
Range query	$O(\log n)$	Only boundary paths are explored
Point update	$O(\log n)$	One root-to-leaf path
Lazy range update	$O(\log n)$	Full covered segments are tagged, not expanded
Memory	$O(n)$	Allocate $4*n$ safely

Final memory hook

A segment tree is an array of interval summaries. Query = ignore / take / split. Update = change leaf, rebuild upward. Lazy update = record whole-segment work, push only when you must.